

Task 5 — Complete Gateway + Discovery Architecture

According to your screenshot, **Task 5** is to integrate the complete flow:

Client

↓

API Gateway

↓

Service Discovery

↓

User Service

Order Service

Payment Service

and then **test the complete flow and document it.**

Step 1 — Make sure these 5 projects exist

You should have:

1. service-registry
2. api-gateway
3. user-service
4. order-service
5. payment-service

The ports can be:

Service Registry → 8761

API Gateway → 8080

User Service → 8081

Order Service → 8083

Payment Service → 8084

Step 2 — Create Payment Service

If you don't already have it, create a Spring Boot project:

Name: payment-service

Artifact: payment-service

Java: 17

Add

Spring Web

Spring Data JPA

Eureka Discovery Client

MySQL Driver

Lombok

Step 3 — Configure Payment Service

Open:

payment-service

→ src/main/resources

→ application.yml

Put,

spring:

application:

name: payment-service

datasource:

url: jdbc:mysql://localhost:3306/payment_db

username: \${DB_USERNAME:root}

password: \${DB_PASSWORD:root}

jpa:

hibernate:

ddl-auto: update

server:

port: 8084

eureka:

client:

service-url:

defaultZone: <http://localhost:8761/eureka/>

The important part is:

name: payment-service

This is the name that Eureka will register for.

Step 4 — Create Payment Entity

Create package:

com.example.paymentservice.entity

Create:

Payment.java

```
package com.example.paymentservice.entity;
```

```
import jakarta.persistence.*;
```

```
import lombok.*;
```

```
@Entity
```

```
@Table(name = "payments")
```

```
@Getter
```

```
@Setter
```

```
@NoArgsConstructor
```

```
@AllArgsConstructor
```

```
public class Payment {
```

```
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    private Long id;
```

```
    private Long orderId;
```

```
    private Double amount;
```

```
    private String status;
```

```
}
```

Step 5 — Create Payment Repository

Create:

com.example.paymentservice.repository

Then:

PaymentRepository.java

```
package com.example.paymentservice.repository;

import com.example.paymentservice.entity.Payment;
import org.springframework.data.jpa.repository.JpaRepository;

public interface PaymentRepository extends JpaRepository<Payment, Long> {
}
```

Step 6 — Create Payment Controller

Create:

```
com.example.paymentservice.controller
```

Then:

```
PaymentController.java
```

Use:

```
package com.example.paymentservice.controller;

import com.example.paymentservice.entity.Payment;
import com.example.paymentservice.repository.PaymentRepository;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/payments")
public class PaymentController {

    private final PaymentRepository paymentRepository;

    public PaymentController(PaymentRepository paymentRepository) {
        this.paymentRepository = paymentRepository;
    }

    @PostMapping
    public Payment createPayment(@RequestBody Payment payment) {
```

```

        return paymentRepository.save(payment);
    }

    @GetMapping("/{id}")
    public Payment getPayment(@PathVariable Long id) {
        return paymentRepository.findById(id)
            .orElseThrow();
    }
}

```

Step 7 — Register Payment Service with Eureka

Open:

PaymentServiceApplication.java

Make sure it has:

```
package com.example.paymentservice;
```

```
import org.springframework.boot.SpringApplication;
```

```
import org.springframework.boot.autoconfigure.SpringBootApplication;
```

```
@SpringBootApplication
```

```
public class PaymentServiceApplication {
```

```
    public static void main(String[] args) {
```

```
        SpringApplication.run(PaymentServiceApplication.class, args);
```

```
    }
```

```
}
```

The Eureka Client dependency plus the Eureka configuration will allow the service to register with the registry.

Step 8 — Update API Gateway

Now open:

api-gateway

→ src/main/resources

→ application.yml

Configure all three services:

spring: application: name: api-gateway

cloud: gateway: routes:

- id: user-service

- uri: lb://user-service

- predicates:

- Path=/api/users/**

- id: order-service

- uri: lb://order-service

- predicates:

- Path=/api/orders/**

- id: payment-service

- uri: lb://payment-service

- predicates:

- Path=/api/payments/**

server: port: 8080

eureka: client: service-url: defaultZone: <http://localhost:8761/eureka/>

Step 9 — Start all services

Start in this order:

Service Registry



User Service



Order Service



Payment Service



API Gateway

Step 10 — Check Eureka

Open your browser:

<http://localhost:8761>

You should see:

Instances currently registered with Eureka

API-GATEWAY

USER-SERVICE

ORDER-SERVICE

PAYMENT-SERVICE

This proves that **service discovery and registration are working**.

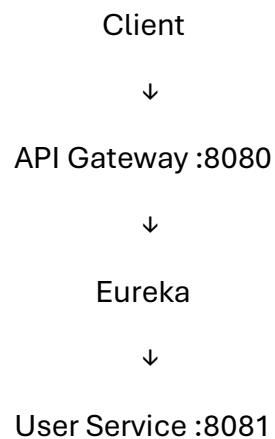
Step 11 — Test through API Gateway

The client should communicate with the **Gateway**, not directly with the individual services.

User

GET <http://localhost:8080/api/users/1>

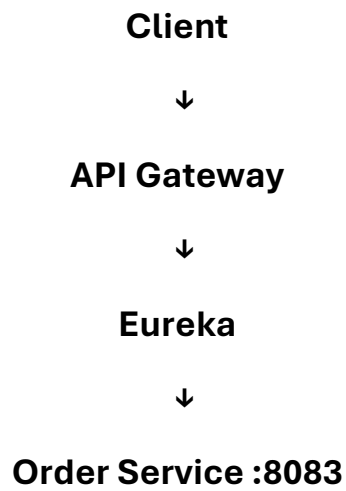
Flow:



Order

GET <http://localhost:8080/api/orders/1>

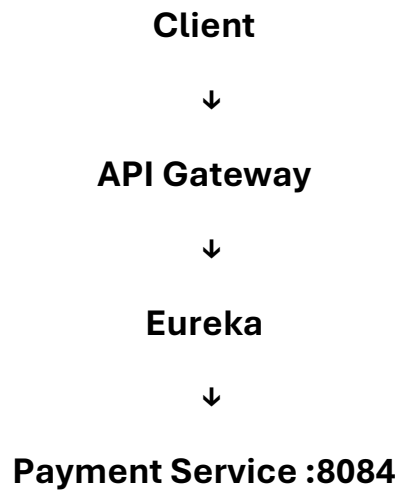
Flow:



Payment

GET <http://localhost:8080/api/payments/1>

Flow:



Step 12 — Test POST Payment

Using Postman, select:

POST

<http://localhost:8080/api/payments>

Choose:

Body → raw → JSON

Send:

```
{  
  
  "orderId": 1,  
  
  "amount": 500.0,  
  
  "status": "SUCCESS"  
}
```

The Request goes

Postman



API Gateway :8080



Eureka

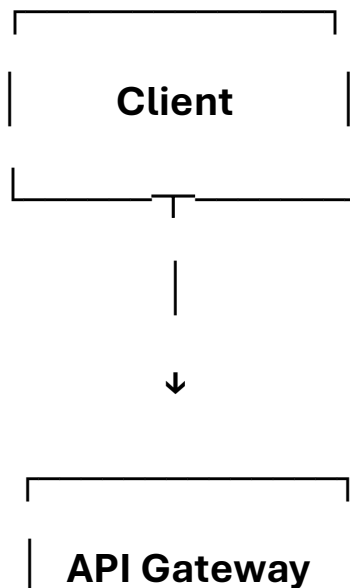


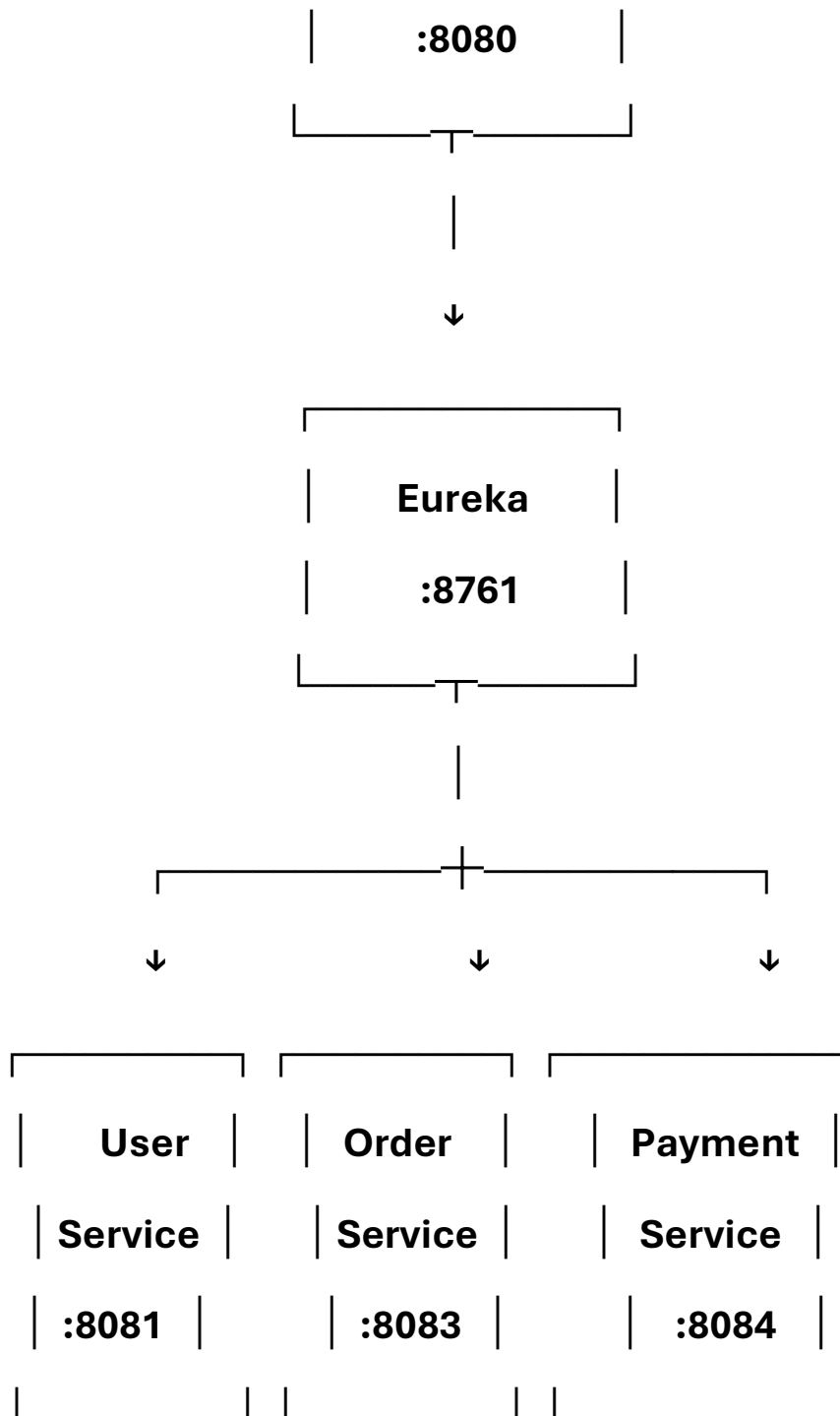
Payment Service :8084



Payment Database

Step 13 — Complete Architecture





Task is completed

When all services appear in Eureka and the User, Order, and Payment requests to work through `localhost:8080` (the API Gateway).

